

**МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ
УНИВЕРСИТЕТ «МИСИС»
ИНСТИТУТ КОМПЬЮТЕРНЫХ НАУК (ИKN)
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ (АСУ)**

Курс «Клиент-серверные приложения и сетевые технологии»

Пояснительная записка к курсовой работе

на тему:

«Разработка клиент-серверного приложения магазина игрушек artlo»

Выполнил: студент группы БИВТ-24-4

Кадыров Артур Рустамович

Подпись: _____

Проверили: Абросимов Никита Андреевич

Шелудяков Пётр Алексеевич

Москва, 2026

Оглавление

1. Предметная область

Предметной областью курсовой работы является разработка персонального клиент-серверного веб-приложения магазина игрушек artlo. Приложение моделирует работу небольшой витрины игрушек: пользователь регистрируется, входит в систему, просматривает товары, добавляет их в корзину, оформляет заказ и видит изменение остатков.

В рамках предметной области выделены две основные группы сущностей: пользователи и товары. Пользователь содержит данные профиля и применяется для демонстрации обязательного API /users. Товар содержит название, категорию, цену, рейтинг, описание, изображение и остаток. Корзина хранится на стороне клиента и связывает пользователя с выбранными товарами.

Практическая проблема, решаемая приложением, состоит в организации удобного доступа к каталогу игрушек и в проверке корректности вводимых данных. Без серверной валидации пользователь мог бы создать профиль с пустым именем, неверным email или слишком коротким паролем. Без разграничения доступа любой пользователь мог бы удалять чужие профили. Поэтому в приложении реализованы регистрация, вход, роль администратора, обработка ошибок и клиентская админ-панель.

Тема выбрана так, чтобы наглядно показать изученные в курсе возможности: HTML/JSP-разметку, CSS-оформление, REST API, LocalStorage, работу с HTTP-запросами и ответами, создание контроллеров и сервисов, использование DTO, декораторов, провайдеров, валидации, логирования и обработки исключений.

2. Постановка задачи

Цель работы — разработать и протестировать персональное клиент-серверное приложение artlo, оформить пояснительную записку по требованиям ГОСТ и подготовить исходный код к загрузке в LMS Moodle и GitHub-репозиторий.

По заданию API должен предоставлять обязательные endpoint: GET /users для получения списка пользователей, GET /users/:id для получения пользователя по идентификатору и POST

/users для создания нового пользователя. Пользователь должен содержать id, name, email и age. Все поля, кроме age, являются обязательными, email должен быть валидным адресом электронной почты.

Данные пользователей по индивидуальной постановке должны храниться в памяти приложения и не сохраняться между перезапусками. Поэтому в проекте используется in-memory хранение в сервисах NestJS. Это решение соответствует формулировке задания, хотя в общем PDF с требованиями к курсовым работам также описывается вариант с БД. В данной работе постоянная БД не используется именно из-за требования о хранении данных в памяти.

Для расширения функциональности и демонстрации темы дополнительно реализованы: регистрация с паролем, вход пользователя по email и паролю, вход администратора, удаление пользователей администратором, получение каталога игрушек, оформление заказа и уменьшение остатков товаров после заказа.

Итоговый проект должен демонстрировать HTML, CSS, Git, LocalStorage, API, управление доступом по категориям пользователей, использование библиотек для валидации данных, создание контроллеров и сервисов, декораторы и провайдеры NestJS, работу с HTTP-запросами и ответами, обработку ошибок и исключений.

3. Описание архитектуры

Приложение построено по клиент-серверной архитектуре. Клиентская часть расположена в папке Source/frontend и реализована на React + Vite. Серверная часть расположена в папке Source/backend и реализована на NestJS. Клиент и сервер запускаются отдельно: сервер на <http://localhost:3000>, клиент на адресе Vite, обычно <http://localhost:5173>.

React-клиент отвечает за интерфейс, регистрацию, вход, отображение товаров, корзину, админ-панель, хранение состояния в LocalStorage и отправку HTTP-запросов. NestJS-сервер отвечает за маршрутизацию запросов, валидацию DTO, работу сервисов, хранение данных в памяти, логирование, обработку исключений и возврат JSON-ответов.

Сервер разделен на модули. UsersModule отвечает за пользователей и обязательные endpoint /users. AuthModule отвечает за вход пользователя и администратора. ToysModule отвечает за каталог игрушек и оформление заказа. AppModule объединяет эти модули в одно приложение.

Логика разделена по слоям: контроллеры принимают HTTP-запросы, сервисы выполняют прикладную логику, DTO описывают входные данные, а фильтр исключений формирует

единый формат ошибок. Такой подход соответствует архитектурным практикам NestJS и упрощает защиту проекта: по каждому endpoint можно показать контроллер, сервис и DTO.

Компонент	Назначение
React + Vite	Клиентский интерфейс, регистрация, вход, каталог, корзина, админ-панель
NestJS API	REST API, валидация, сервисы, обработка ошибок, логирование
UsersModule	GET /users, GET /users/:id, POST /users, DELETE /users/:id
AuthModule	POST /auth/login для входа пользователя и администратора
ToysModule	GET /toys и POST /toys/checkout для каталога и заказа
LocalStorage	Хранение сессии и корзины на стороне клиента

Таблица 1 — 3. архитектуры

4. Описание структуры БД

В работе используется учебная SQLite-база данных, создаваемая в памяти backend-приложения при запуске. Такой вариант позволяет показать полноценную структуру БД, SQL-таблицы и связи между сущностями, но при этом не нарушает условие индивидуального задания: данные не сохраняются между перезапусками приложения.

В базе данных реализовано пять таблиц: roles, users, toys, orders и order_items. Таблица users хранит пользователей с полями id, name, email, password_hash, age, role_id и created_at. Таблица roles хранит категории доступа: client и admin. Таблица toys хранит каталог товаров, цену, категорию, рейтинг, изображение и остаток stock.

Таблица orders хранит оформленные заказы: id, user_id, customer_email, total, status и created_at. Таблица order_items хранит состав заказа и связывает заказ с товарами через order_id и toy_id. Таким образом, структура БД содержит больше четырех таблиц и показывает связи один-ко-многим: один пользователь может иметь несколько заказов, один заказ содержит несколько позиций.

Корзина хранится в LocalStorage на стороне клиента под ключом toy-shelf-cart. При оформлении заказа клиент отправляет товары на сервер через POST /toys/checkout. Сервер в транзакции проверяет остатки в таблице toys, уменьшает stock, создает запись в orders и добавляет строки в order_items.

Сущность	Поля	Назначение
User	id, name, email, passwordHash, age	Профиль пользователя; passwordHash не возвращается клиенту
Toy	id, title, category, price, available, stock, rating, description, image	Карточка товара каталога
Cart	toyId, quantity	Корзина клиента в LocalStorage

Таблица 2 — 4. структуры БД

4.1. Таблицы базы данных

roles — справочник ролей пользователей. Используется для разделения обычных клиентов и администратора.

users — таблица пользователей. Содержит обязательные поля id, name и email, необязательное поле age, хеш пароля password_hash и ссылку role_id на таблицу roles.

toys — таблица товаров магазина. Хранит название, категорию, цену, рейтинг, описание, путь к изображению, признак доступности и остаток stock.

orders — таблица заказов. Хранит идентификатор заказа, ссылку на пользователя, email покупателя, итоговую сумму, статус и дату создания.

order_items — таблица позиций заказа. Хранит связь заказа с товарами, количество и цену товара на момент покупки.

Связи между таблицами реализованы через внешние ключи: users.role_id связан с roles.id, orders.user_id связан с users.id, order_items.order_id связан с orders.id, а order_items.toy_id связан с toys.id.

5. Описание серверной части

Серверная часть разработана на NestJS. В файле main.ts создается приложение через NestFactory, включается CORS, подключается глобальный ValidationPipe, регистрируется HttpExceptionHandler и RequestLoggerMiddleware. Сервер запускается на порту 3000.

UsersController реализует endpoint GET /users, GET /users/:id, POST /users и DELETE /users/:id. GET /users возвращает список публичных профилей. GET /users/:id ищет пользователя по id.

POST /users создает нового пользователя после проверки DTO. DELETE /users/:id используется администратором для удаления пользователя.

UserService работает с таблицей users через DatabaseService. При создании пользователя сервис проверяет уникальность email SQL-запросом, хеширует пароль bcrypt, записывает пользователя в таблицу users и возвращает публичную версию профиля без password_hash.

CreateUserDto задает правила валидации: name должен быть непустой строкой, email должен быть валидным адресом электронной почты, password должен иметь длину не меньше 6 символов, age является необязательным целым числом от 1 до 120. Для валидации используется class-validator, для преобразования age в число используется class-transformer.

AuthController содержит POST /auth/login. AuthService проверяет вход администратора по логину admin и паролю admin123, а обычного пользователя — по email и паролю. Для проверки пароля используется bcrypt.compareSync. В ответ возвращаются token, role, login и публичные данные пользователя.

ToysController содержит GET /toys и POST /toys/checkout. GET /toys возвращает каталог товаров. POST /toys/checkout принимает массив товаров из корзины, проверяет количество и остаток, уменьшает stock, обновляет available и возвращает сумму заказа вместе с обновленным каталогом.

HttpExceptionFilter формирует единый формат ошибок JSON: statusCode, timestamp и details. Это удобно для frontend, потому что все ошибки можно обработать одинаково. RequestLoggerMiddleware фиксирует метод, URL, статус ответа и время выполнения запроса, что помогает при тестировании и защите.

С точки зрения безопасности реализованы базовые учебные меры: пароль не хранится открытым текстом, passwordHash не отдается клиенту, email валидируется, лишние поля отбрасываются через ValidationPipe, а административная функция удаления пользователей показывается только после входа администратора. Для промышленного проекта можно дополнительно добавить JWT, guards и серверную проверку ролей на каждом защищенном endpoint.

Endpoint	Метод	Назначение
/users	GET	Получение списка пользователей
/users/:id	GET	Получение пользователя по id
/users	POST	Регистрация пользователя

/users/:id	DELETE	Удаление пользователя администратором
/auth/login	POST	Вход пользователя или администратора
/toys	GET	Получение каталога товаров
/toys/checkout	POST	Оформление заказа и уменьшение остатков

Таблица 3 — 5. серверной части

6. Описание клиентской части

Клиентская часть разработана на React + Vite. Основной файл App.jsx содержит состояние текущей сессии, формы входа, формы регистрации, списка пользователей, списка товаров, корзины, выбранной категории и сообщения об ошибке. API-запросы вынесены в api.js, работа с LocalStorage — в storage.js, оформление — в styles.css.

Стартовая страница содержит два режима: вход и регистрация. В режиме регистрации пользователь вводит имя, email, пароль и возраст. После отправки формы выполняется POST /users. В режиме входа пользователь выбирает тип входа: пользователь или администратор. Пользователь входит по email и паролю, администратор — по admin/admin123.

После входа открывается магазин artlo. В верхней части отображается бренд, название «Магазин игрушек» и карточка текущего пользователя. Ниже показана статистика: количество товаров, количество товаров в корзине и сумма заказа.

Каталог состоит из пяти карточек товаров с фотографиями, названием, категорией, рейтингом, описанием, ценой и остатком. Фильтр категорий позволяет показать только товары выбранной категории. Изображения товаров хранятся локально в Source/frontend/public/images, поэтому сайт не зависит от внешних картинок.

Корзина находится справа от каталога. Пользователь может добавлять товары, уменьшать и увеличивать количество, удалять позицию из корзины и оформить заказ. Корзина сохраняется в LocalStorage, поэтому выбранные товары не исчезают после обновления страницы. При оформлении заказа frontend отправляет POST /toys/checkout, очищает корзину и обновляет остатки товаров.

Администратор после входа видит блок «Управление пользователями». В нем выводится список пользователей и кнопки удаления. При нажатии на кнопку выполняется DELETE /users/:id, после чего список обновляется через GET /users.

CSS-оформление сделано так, чтобы сайт не выглядел стандартной белой страницей: используется цветной фон, темная брендовая стартовая панель, акцентные кнопки, карточки с тенями, цветовые бейджи остатков и адаптивная сетка. Фотография куклы отображается через object-fit: contain, поэтому вертикальное изображение не обрезается.

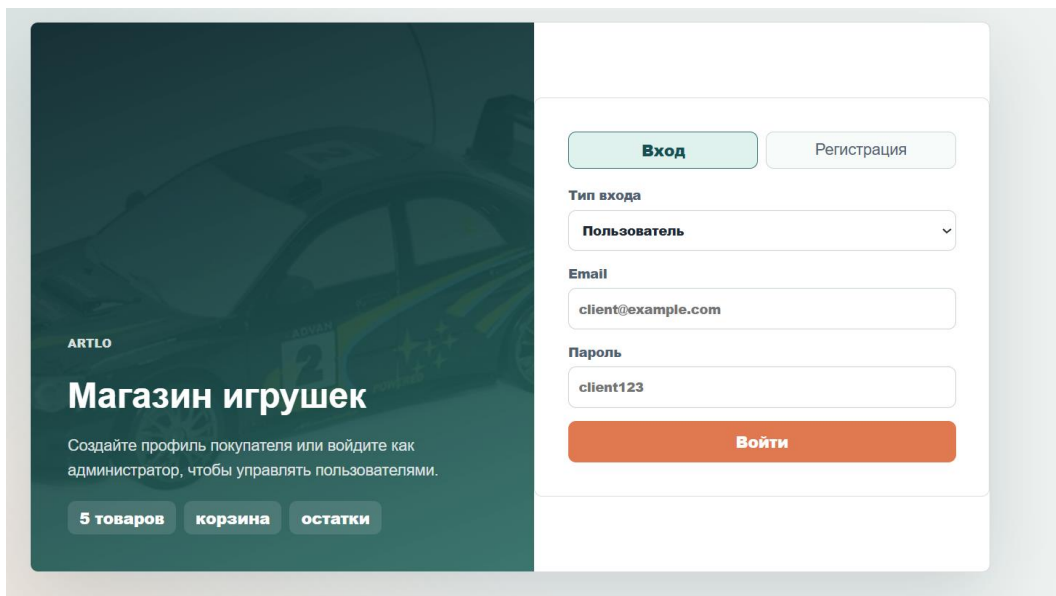


Рисунок 1 — Стартовая страница входа и регистрации

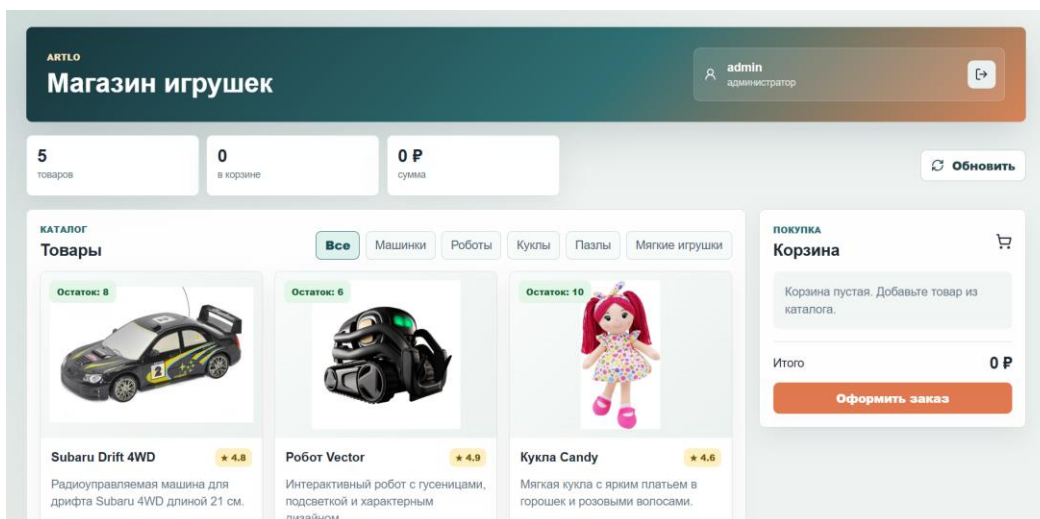


Рисунок 2 — Главная страница магазина artlo

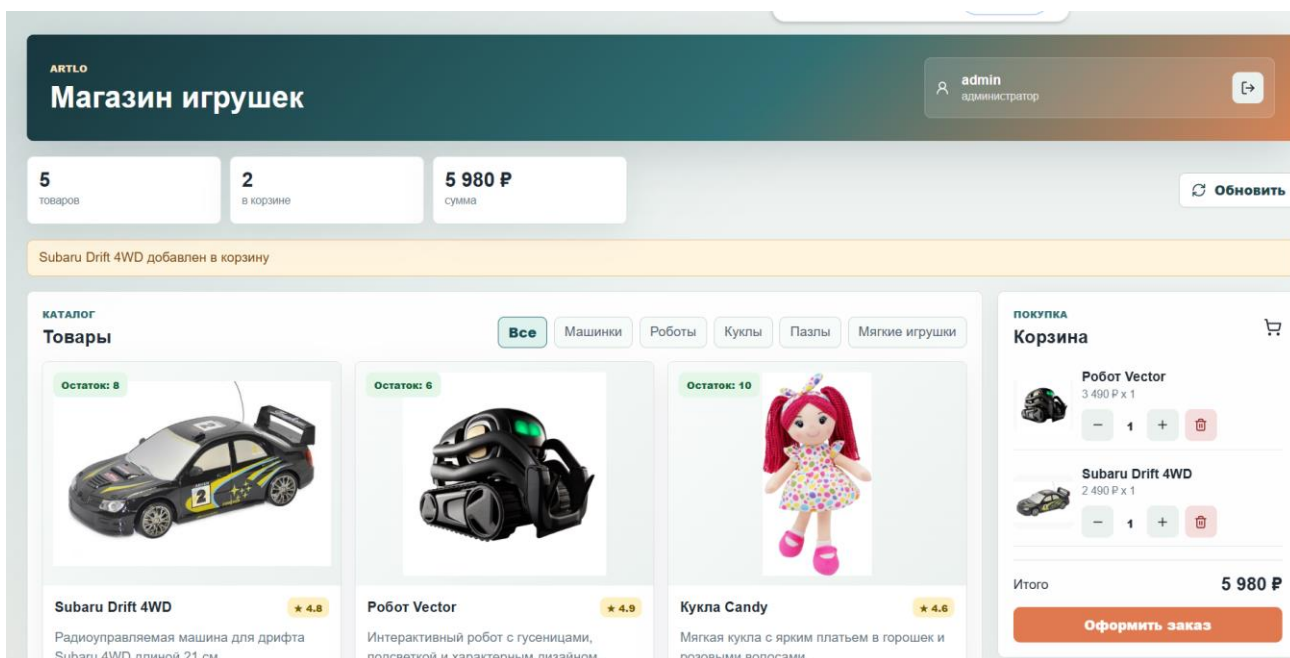


Рисунок 3 — Корзина и оформление заказа

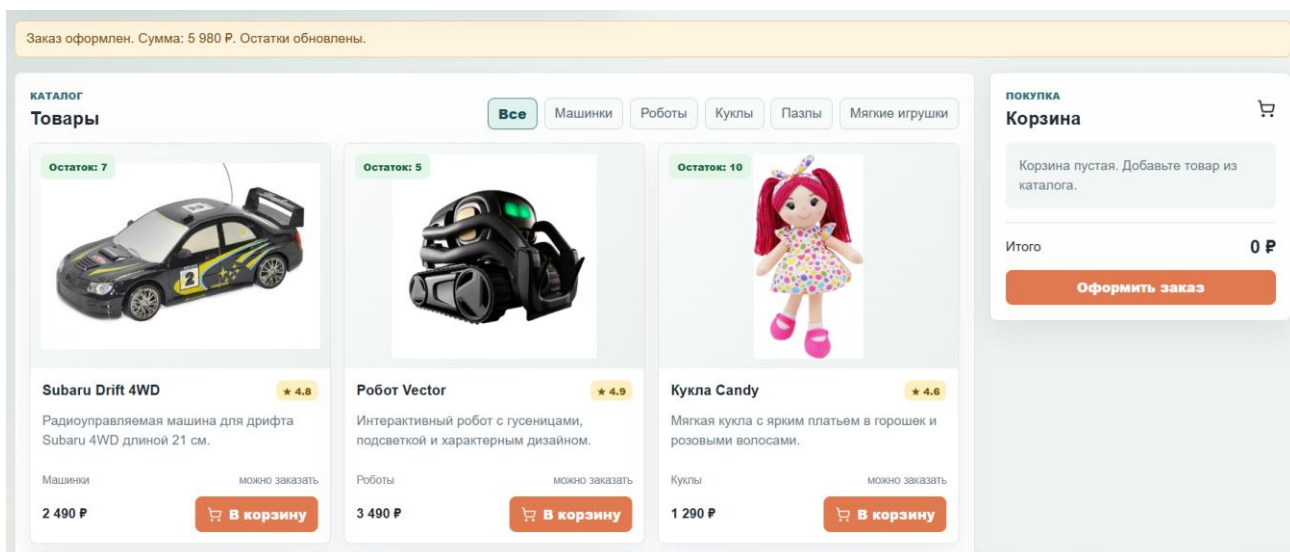


Рисунок 4 — Изменение остатка товара после заказа

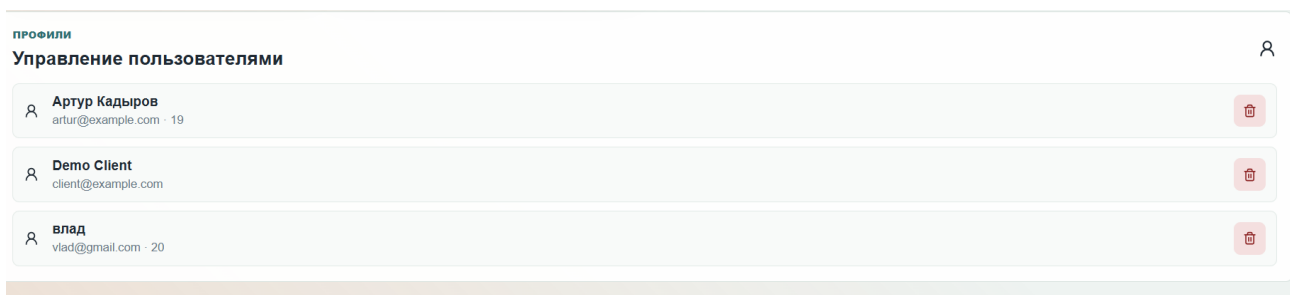


Рисунок 5 — Админ-панель удаления пользователей



Рисунок 9 — Ответ GET /users или GET /toys

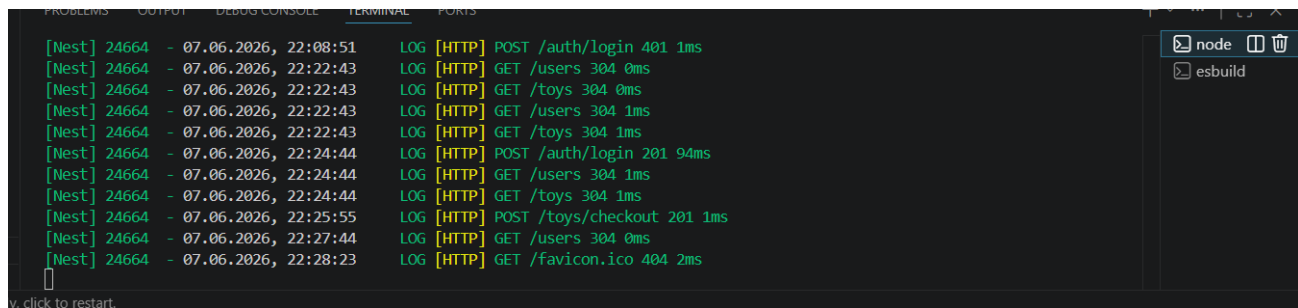


Рисунок 7 — Логи HTTP-запросов backend

7. Пояснение основных фрагментов программного кода

Данный раздел добавлен для пояснения листинга программы на защите. В нем кратко описано, какие файлы являются основными, как они связаны между собой и как проходит обработка данных от формы на сайте до ответа сервера.

7.1. Основные файлы backend

Серверная часть находится в папке Source/backend/src. Главной точкой входа является main.ts. В этом файле создается NestJS-приложение, включается CORS для связи с React-клиентом, подключается глобальная валидация DTO, общий фильтр ошибок и логирование HTTP-запросов.

```
app.enableCors();
app.useGlobalPipes(new ValidationPipe({ whitelist: true, transform: true }));
app.useGlobalFilters(new HttpExceptionFilter());
app.use(new RequestLoggerMiddleware().use);
await app.listen(3000);
```

Файл app.module.ts собирает приложение из модулей UsersModule, AuthModule и ToysModule. Такое разделение показывает архитектуру NestJS: каждый модуль отвечает за свою часть предметной области, а контроллеры и сервисы подключаются через провайдеры.

7.2. Пользователи, регистрация и вход

Модуль users реализует обязательные endpoint задания: GET /users, GET /users/:id и POST /users. Дополнительно добавлен DELETE /users/:id для администратора, чтобы показать управление доступом и работу админ-панели.

```
@Get()
findAll() { return this.usersService.findAll(); }

@Get('/:id')
findOne(@Param('id') id: string) { return this.usersService.findOne(id); }

@Post()
create(@Body() dto: CreateUserDto) { return this.usersService.create(dto); }
```

В UsersService данные пользователей читаются и записываются через SQL-таблицу users. При этом база создается в памяти при запуске backend, поэтому после перезапуска данные возвращаются к начальному демонстрационному состоянию. Это согласует требование задания о несохранении данных с требованием показать структуру БД.

Вход вынесен в AuthModule. Обычный пользователь входит по email и паролю, а администратор входит по учебной учетной записи admin / admin123. После успешной проверки frontend получает объект с ролью пользователя. По этой роли интерфейс решает, показывать обычный магазин или еще и блок управления пользователями.

7.3. Валидация, безопасность и ошибки

Валидация данных выполняется через class-validator в DTO-классах. Например, в CreateUserDto поля name, email и password обязательны, email проверяется как адрес электронной почты, а age является необязательным числовым полем. Благодаря ValidationPipe некорректные данные отбрасываются до попадания в сервис.

```
@IsString()
@NotEmpty()
name: string;

@IsEmail()
email: string;

@IsOptional()
@Type( () => Number)
```

```
@IsInt()  
age?: number;
```

Безопасность в проекте реализована на учебном уровне: пароль не хранится открытым текстом, а преобразуется в bcrypt-хеш; passwordHash не возвращается в ответах API; вход проверяет пару email/password; роли разделяют обычного клиента и администратора. Для реального промышленного проекта дополнительно потребовались бы JWT-токены, HTTPS, хранение пользователей в базе данных и полноценная серверная проверка прав на каждый защищенный endpoint.

Ошибки обрабатываются централизованно через HttpExceptionHandler. Если пользователь не найден, email уже занят или остатка товара не хватает, backend возвращает понятный JSON-ответ с кодом ошибки, временем и деталями. Frontend показывает это сообщение пользователю.

7.4. Каталог товаров, корзина и остатки

Модуль toys отвечает за витрину магазина и работу заказов. GET /toys читает товары из таблицы toys. POST /toys/checkout принимает содержимое корзины, проверяет остатки, уменьшает stock и записывает заказ в таблицы orders и order_items. Дополнительно endpoint GET /toys/orders позволяет показать историю заказов и доказать работу базы данных.

```
const    toy      =    this.toys.find((item)    =>    item.id    ===    orderItem.id);  
if        (!toy    ||    toy.stock    <    orderItem.quantity)    {  
    throw    new    BadRequestException('Недостаточно    товара    на    складе');  
}  
toy.stock    -=    orderItem.quantity;  
toy.available = toy.stock > 0;
```

Главная идея обработки заказа такая: клиент отправляет id товара и количество, сервер проверяет наличие товара и достаточный остаток, затем уменьшает stock. После успешного заказа frontend повторно получает список товаров и на экране видно, что остаток уменьшился, например с 14 до 13.

7.5. Основные файлы frontend

Клиентская часть находится в папке Source/frontend/src. Основной файл App.jsx содержит состояние приложения: текущего пользователя, список товаров, корзину, форму входа, форму регистрации и режим администратора. Компонент отправляет запросы к backend через функции из api.js.

Файл `api.js` содержит базовый адрес backend `http://localhost:3000` и общую функцию `request`. Она отправляет HTTP-запрос, преобразует JSON-ответ и выбрасывает ошибку, если сервер вернул неуспешный статус. Благодаря этому в `App.jsx` не дублируется одинаковая логика `fetch`.

Файл `storage.js` отвечает за `LocalStorage`. В нем сохраняются данные текущей сессии и корзина. Это нужно, чтобы после обновления страницы выбранные товары и пользовательская сессия не исчезали сразу на клиентской стороне.

```
localStorage.setItem(CART_KEY, JSON.stringify(cart));
const savedCart = JSON.parse(localStorage.getItem(CART_KEY) || '[]');
localStorage.setItem(SESSION_KEY, JSON.stringify(user));
```

7.6. Как соединены frontend и backend

Связь частей приложения выполняется через REST API. Backend запускается на порту 3000, а frontend через Vite обычно запускается на порту 5173. Когда пользователь регистрируется, React отправляет POST `/users`. Когда пользователь входит, React отправляет POST `/auth/login`. Когда открывается магазин, React отправляет GET `/toys`. При оформлении корзины отправляется POST `/toys/checkout`.

База данных в проекте есть, но она работает в режиме in-memory SQLite. Это значит, что таблицы `roles`, `users`, `toys`, `orders` и `order_items` создаются при запуске сервера, используются обычными SQL-запросами во время работы приложения и очищаются после остановки backend.

Если на защите нужно кратко объяснить проект, можно сказать так: frontend отображает интерфейс и отправляет запросы, backend принимает запросы в контроллерах, передает работу сервисам, сервисы проверяют данные и меняют in-memory массивы, после чего backend возвращает JSON-ответ обратно на сайт.

8. Заключение

В результате выполнения курсовой работы разработано клиент-серверное приложение `artlo`. Приложение реализует обязательные endpoint `/users`, хранит данные в памяти, проверяет обязательные поля, валидирует email, использует пароль при регистрации и поддерживает вход пользователя.

Серверная часть демонстрирует NestJS-модули, контроллеры, сервисы, DTO, декораторы, провайдеры, ValidationPipe, class-validator, class-transformer, bcryptjs, обработку исключений и логирование запросов. Клиентская часть демонстрирует HTML/JSX, CSS, Fetch API, LocalStorage, регистрацию, вход, корзину, оформление заказа, обновление остатков и админское удаление пользователей.

Работа приложения проверяется через браузер и HTTP-запросы. Основные сценарии: регистрация пользователя, вход пользователя, вход администратора, получение списка пользователей, удаление пользователя администратором, получение каталога товаров, добавление товара в корзину, оформление заказа и уменьшение остатка.

Проект опубликован в GitHub-репозитории.

9. Список литературы

1. NestJS Documentation [Электронный ресурс]. URL: <https://docs.nestjs.com/> (дата обращения: 06.06.2026).
2. React Documentation [Электронный ресурс]. URL: <https://react.dev/> (дата обращения: 06.06.2026).
3. Vite Documentation [Электронный ресурс]. URL: <https://vite.dev/> (дата обращения: 06.06.2026).
4. bcryptjs. npm package [Электронный ресурс]. URL: <https://www.npmjs.com/package/bcryptjs> (дата обращения: 06.06.2026).
5. Репозиторий исходного кода приложения artlo [Электронный ресурс]. URL: <https://github.com/Artlogg/toy-shelf-coursework> (дата обращения: 06.06.2026).